Fall 2025

KAIST EE412

Foundation of Big Data Analytics

Final Exam

Name: _____

Student ID: _____

I agree to comply with the School of Electrical Engineering Honor Code.

Signature: _____

This exam is open notes, closed book, and closed computer. There are 22 pages in total. You have 150 minutes (13:00 – 15:30) to complete your exam. Please use this exam sheet to write answers. You may write in English or Korean.

Read the questions carefully and focus your answers on what has been asked. You are allowed to ask the instructor/TAs for help only in understanding the questions, in case you find them not completely clear. Be concise and precise in your answers and state clearly any assumption you may have made. Good luck!

Question	Score
1	/24
2	/20
3	/20
4	/20
5	/20
6	/20
Total	/124

1 (24 points) TRUE/FALSE

State if the following statements are true or false. Please write TRUE or FALSE in the space provided. Each problem is worth 1 point, but you will be penalized -1 point for an incorrect answer, so leave the answer blank if you don't know.

(a) The "long tail" phenomenon refers to physical stores being able to customize their limited shelf space to each individual customer.

Answer: FALSE

(b) A utility matrix in recommender systems is usually dense because most users rate most items in the catalog.

Answer: FALSE

(c) In UV decomposition for recommender systems, we can safely apply standard SVD directly to the sparse rating matrix by treating all missing ratings as zeros.

Answer: FALSE

(d) L2 regularization in latent factor models tends to pull user and item vectors toward the origin, so only strong factors remain large.

Answer: TRUE

(e) PageRank treats a page with many out-going links but no in-coming links as highly important.

Answer: FALSE

(f) Google's use of probabilistic teleportation in the Google Matrix is designed so that both dead ends and spider traps are handled at once.

Answer: TRUE

(g) In approximate personalized PageRank based on a lazy random walk, the approximation error tends to grow for nodes that are farther away from the seed node.

Answer: TRUE

(h) In the spammer's model of the web, accessible pages are defined as pages that are completely controlled by the spammer, possibly across multiple domain names.

Answer: FALSE

(i) Activation functions are crucial because, without them, stacking multiple layers would still only represent a linear transformation overall.

Answer: TRUE

(j) Inductive bias is something that always improves generalizability.

Answer: FALSE

(k) For binary classification, mean squared error (MSE) is always preferred over cross-entropy loss because it guarantees faster convergence.

Answer: FALSE

(l) The cross-entropy between the true distribution p and the model distribution q is minimized when $q = p$.

Answer: TRUE

(m) A good embedding is typically dense, low-dimensional, and able to capture semantic similarity between items.

Answer: TRUE

(n) An autoencoder is a neural network trained to reconstruct its input, and SVD can be interpreted as a kind of linear autoencoder.

Answer: TRUE

(o) Hierarchical softmax and negative sampling are introduced to reduce the computational cost of training models with a full softmax over all nodes.

Answer: TRUE

(p) Node2vec uses biased random walks so that, by tuning hyperparameters, it can behave more like BFS or more like DFS on the graph.

Answer: TRUE

(q) DeepWalk and node2vec can handle inductive generalization to unseen nodes.

Answer: FALSE

(r) One potential advantage of deep graph encoders is that they can share parameters across different nodes, reducing the total number of parameters.

Answer: TRUE

(s) Compared to NGCF, LightGCN introduces additional learnable weight matrices at each layer to model more complex non-linear transformations.

Answer: FALSE

(t) A k-layer GNN generates node embeddings using information from k-hop neighbors, and this process can be represented as a computational graph of length k.

Answer: TRUE

(u) A Bloom filter may return false positives, but it is designed so that it never returns false negatives.

Answer: TRUE

(v) In reservoir sampling, elements that arrive later are kept with higher probability than elements that appeared earlier.

Answer: FALSE

(w) A key intuition behind the Flajolet–Martin algorithm is that for a good hash function, the probability that a hash value ends with at least r zeros is 2^{-r}.

Answer: TRUE

(x) In the Flajolet–Martin algorithm, each hashed element is represented as a bit string, and the algorithm records the position of the leftmost 1-bit in that string.

Answer: FALSE

2 [20 points] Recommender Systems

Let's consider a recommendation with implicit feedback, where the utility matrix is given as $\mathbf{R} \in \{0,1\}^{n \times m}$ with n users and m items. This can be viewed as a bipartite undirected graph $\mathcal{G}$ consisting of:

- nodes V: $x \in V$ if $x \in U$ or $x \in I$, where U and I are the sets of users and items.
- edges E: $(x, y) \in E$ if $R_{xy} = 1$ or $R_{yx} = 1$.

In collaborative filtering, there are various ways to measure similarity. In this problem, we explore a graph-based similarity called **SimRank** ($\mathrm{SR}(\cdot,\cdot) \in [0,1]$) defined by:

$$\mathrm{SR}(x,y) = \begin{cases} 1, & x = y \\ \dfrac{c}{|N(x)||N(y)|} \displaystyle\sum_{z \in N(x)} \sum_{w \in N(y)} \mathrm{SR}(z,w), & x \neq y \end{cases}$$

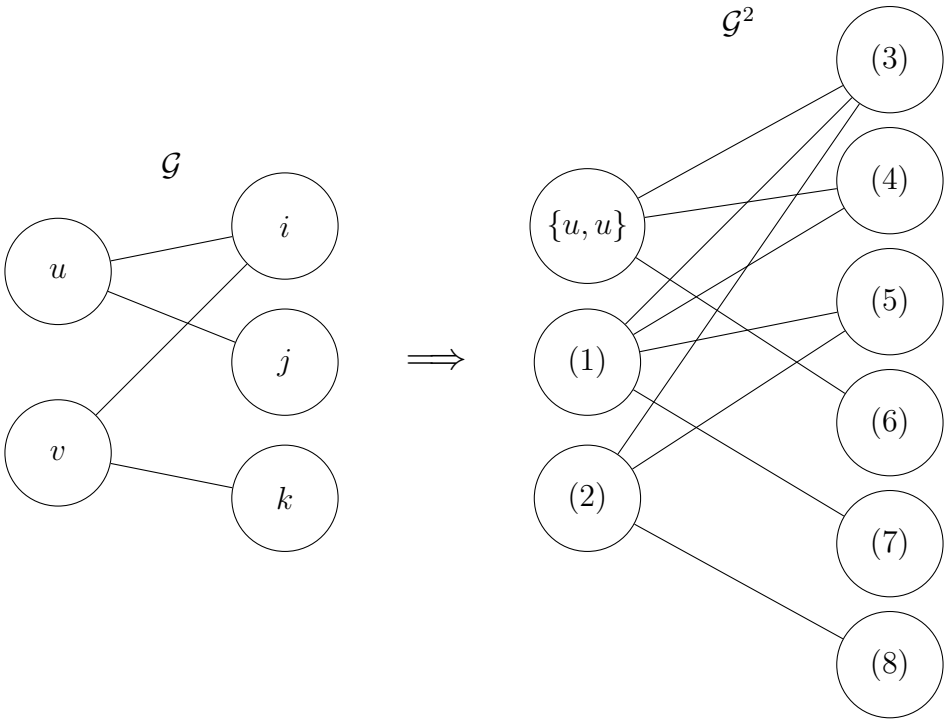
where $c \in (0,1)$ is a decay factor and $N(x)$ is the set of neighbors of x in $\mathcal{G}$.

(a) [8 points] To compute SimRank, consider the undirected graph $\mathcal{G}^2$ consisting of:

- nodes V^2: $\{p,q\} \in V^2$ if $p \in V$, $q \in V$ and $\mathrm{SR}(p,q) > 0$.
- edges E^2: $(\{p,q\},\{s,t\}) \in E^2$ if $((p,s) \in E$ and $(q,t) \in E)$ or $((p,t) \in E$ and $(q,s) \in E)$.

Suppose that $\mathcal{G}$ is given as follows. Fill in the node labels on the numbered circles (1)–(8) in $\mathcal{G}^2$ based on the given information, i.e., $\{u,u\}$ in the upper left node.

Note: $\{p,q\} = \{q,p\}$.



Answer.

(1) $\{u,v\}$ (2) $\{v,v\}$ (3) $\{i,i\}$ (4) $\{i,j\}$ (5) $\{i,k\}$ (6) $\{j,j\}$ (7) $\{j,k\}$ (8) $\{k,k\}$

Criterion.

- 1 point for each.

(b) [3 points] Define the transition matrix $\mathbf{P} \in \mathbb{R}^{(n+m)\times(n+m)}$ as follows:

$$P_{xy} = \begin{cases} 1/|N(y)|, & (x,y) \in E \\ 0, & (x,y) \notin E \end{cases}$$

Let $\mathbf{S} \in \mathbb{R}^{(n+m)\times(n+m)}$ be the SimRank matrix ($S_{xy} = \mathrm{SR}(x,y)$) and let $\vee$ denote the entry-wise maximum, i.e., $(\mathbf{A} \vee \mathbf{B})_{xy} = \max(A_{xy}, B_{xy})$.

Show that $\mathbf{S} = (c\mathbf{P}^\top\mathbf{S}\mathbf{P}) \vee \mathbf{I}$, where $\mathbf{I} \in \mathbb{R}^{(n+m)\times(n+m)}$ is the identity matrix.

Answer.

First expand the (x,y)-entry of $\mathbf{P}^\top\mathbf{S}\mathbf{P}$:

$$(\mathbf{P}^\top\mathbf{S}\mathbf{P})_{xy} = \sum_z \sum_w P_{zx}\, S_{zw}\, P_{wy}.$$

By the definition of $\mathbf{P}$, $P_{zx} = \frac{1}{|N(x)|}$ if $z \in N(x)$, 0 otherwise. Similarly, $P_{wy} = \frac{1}{|N(y)|}$ if $w \in N(y)$, and 0 otherwise. Therefore,

$$\begin{aligned}(\mathbf{P}^\top\mathbf{S}\mathbf{P})_{xy} &= \sum_{z\in N(x)} \sum_{w\in N(y)} \frac{1}{|N(x)|} S_{zw} \frac{1}{|N(y)|} \\ &= \frac{1}{|N(x)|\,|N(y)|} \sum_{z\in N(x)} \sum_{w\in N(y)} S_{zw}.\end{aligned}$$

For $x \neq y$, the SimRank definition says

$$S_{xy} = \mathrm{SR}(x,y) = \frac{c}{|N(x)|\,|N(y)|} \sum_{z\in N(x)} \sum_{w\in N(y)} \mathrm{SR}(z,w) = c\,(\mathbf{P}^\top\mathbf{S}\mathbf{P})_{xy}.$$

For $x = y$, SimRank enforces $S_{xx} = 1$ for all x. Collecting these conditions in matrix form, we can write:

$$\mathbf{S} = (c\,\mathbf{P}^\top\mathbf{S}\mathbf{P}) \vee \mathbf{I},$$

Criterion.

- Deduct points if there is a minor mistake.

(c) [9 points] Define the sequence $\{\mathbf{S}^{(k)}\}$ as follows:

$$\mathbf{S}^{(0)} = \mathbf{I}, \qquad \mathbf{S}^{(k+1)} = (c\,\mathbf{P}^\top \mathbf{S}^{(k)} \mathbf{P}) \vee \mathbf{I}, \quad k = 0, 1, 2, \ldots$$

Do the following steps:

(i) Show that each entry of $\mathbf{S}^{(k)}$ is bounded by induction.
(ii) Show that each entry of $\mathbf{S}^{(k)}$ forms a monotone sequence by induction.
(iii) If two solutions $\mathbf{S}'$ and $\mathbf{S}''$ exist for $\mathbf{S} = (c\mathbf{P}^\top \mathbf{S}\mathbf{P}) \vee \mathbf{I}$, show that $\mathbf{S}' = \mathbf{S}''$.

which shows that $\mathbf{S}$ exists and is unique.

Answer.

We write $S_{xy}^{(k)}$ for the (x, y)-entry of $\mathbf{S}^{(k)}$. Then, for $x = y$, $S_{xx}^{(k+1)} = 1$, and for $x \neq y$ we can rewrite

$$S_{xy}^{(k+1)} = \frac{c}{|N(x)|\,|N(y)|} \sum_{z \in N(x)} \sum_{w \in N(y)} S_{zw}^{(k)}.$$

We claim that $0 \leq S_{xy}^{(k)} \leq S_{xy}^{(k+1)} \leq 1$ for all $x \neq y$ and $k \geq 0$ holds by induction.

- **Boundness**: It holds for $k = 0$. Assume $0 \leq S_{xy}^{(k)} \leq 1$. Then, $S_{xy}^{(k+1)} \geq 0$ and

$$S_{xy}^{(k+1)} \leq \frac{c}{|N(x)|\,|N(y)|} \sum_{z \in N(x)} \sum_{w \in N(y)} 1 = \frac{c}{|N(x)|\,|N(y)|} \cdot |N(x)||N(y)| = c < 1.$$

- **Monotonicity**: It holds for $k = 0$ since $S_{xy}^{(0)} = 0$ and $S_{xy}^{(1)} \geq 0$. Assume $S_{zw}^{(k-1)} \leq S_{zw}^{(k)}$ for all z, w. Then

$$S_{xy}^{(k)} = \frac{c}{|N(x)|\,|N(y)|} \sum_{z \in N(x)} \sum_{w \in N(y)} S_{zw}^{(k-1)} \leq S_{xy}^{(k+1)} = \frac{c}{|N(x)|\,|N(y)|} \sum_{z \in N(x)} \sum_{w \in N(y)} S_{zw}^{(k)}.$$

Thus, it converges to a limit $\mathbf{S}'$ by the completeness axiom of calculus. Then,

$$\lim_{k \to \infty} \mathbf{S}^{(k+1)} = \lim_{k \to \infty} (c\,\mathbf{P}^\top \mathbf{S}^{(k)} \mathbf{P}) \vee \mathbf{I} \quad \Rightarrow \quad \mathbf{S}' = (c\,\mathbf{P}^\top \mathbf{S}' \mathbf{P}) \vee \mathbf{I},$$

which shows that $\mathbf{S}'$ is a solution $\mathbf{S}$ to the SimRank equation.

For **uniqueness**, suppose $\mathbf{S}'$ and $\mathbf{S}''$ are two solutions, i.e.,

$$\mathbf{S}' = (c\,\mathbf{P}^\top \mathbf{S}' \mathbf{P}) \vee \mathbf{I}, \qquad \mathbf{S}'' = (c\,\mathbf{P}^\top \mathbf{S}'' \mathbf{P}) \vee \mathbf{I}.$$

For all x, y, define $\delta(x, y) = S'_{xy} - S''_{xy}$ and $M = \max_{x,y} |\delta(x, y)|$. We claim that $M = 0$. Note that $S'_{xx} = S''_{xx} = 1$. For $x \neq y$,

$$\delta(x, y) = c \sum_z \sum_w P_{zx}\, \delta(x, y)\, P_{wy}.$$

Taking absolute values and using $|\delta(x, y)| \leq M$,

$$|\delta(x, y)| \leq c \sum_z \sum_w P_{zx} |\delta(x, y)|\, P_{wy} \leq cM \sum_z \sum_w P_{zx} P_{wy} = cM.$$

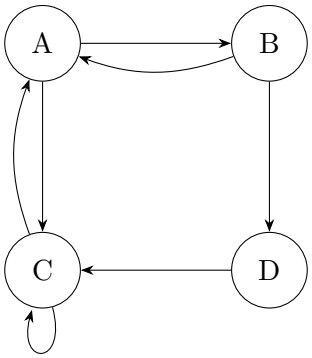
If we choose (x, y) which achieves $|\delta(x, y)| = M$, it yields $M \leq cM$. Since $0 < c < 1$, this implies $M = 0$ so that $\mathbf{S}' = \mathbf{S}''$.

Criterion.

- 3 points for each.

3 [20 points] Link Analysis

(a) [4 points] Consider the following web graph of four pages A, B, C, and D.



Compute the PageRank values of all nodes after one iteration using teleportation probability $1 - \beta = 0.2$ with $\beta = 0.8$. The initial vector is uniform.

[0.25, 0.15, 0.45, 0.15]

(b) [6 points] A spammer attempts to inflate the PageRank of page C in the graph shown in (a) by creating a link farm. The spammer adds new M pages pointing page C.

(i) Compute one iteration of TrustRank on page A, B, and C using $\beta = 0.8$ assuming only page B is a trusted page.

[0.4,0.2,0]

(ii) Compute the spam mass of page A, B, and C.

[-0.6-M, -(1+M)/3, 1]

(c) [10 points] Consider a huge web graph where we compute the PageRank vector utilizing power iteration: $r^{new} = M \cdot r^{old}$. Here M is the transition matrix and r is the rank vector. Let $|M|$ and $|r|$ denote the storage sizes of the matrix M and the rank vector r on disk, respectively.

Assume that:

- $|M| \gg |r^{old}|, |r^{new}|$
- The main memory is limited to store the entire r^{new} vector at once. So we divide r^{new} into k equal-sized blocks such that each block fits into the memory.
- To avoid inefficient random disk I/O, the active block of r^{new} being computed must reside in the **main memory**. Once a block is fully computed, it is written back to the disk sequentially.
- The total I/O cost is determined solely by the amount of data read from or written to the **disk** (memory operations are negligible).

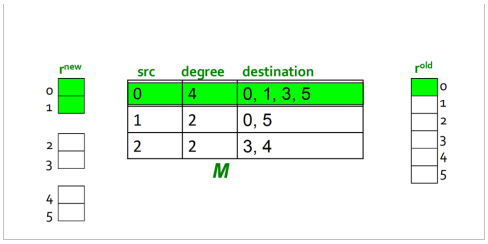


Figure 1: Basic Block-Based Update

(i) To compute a single block r_i^{new} in Figure 1, we must scan the entire matrix M and the source vector r^{old} from the disk. Considering that this process should be repeated for all k blocks, derive the total I/O cost per iteration in terms of $|M|$, $|r^{old}|$, $|r^{new}|$, and k.

$\boldsymbol{k(|M| + |r^{old}|) + |r^{new}|}$

(ii) To improve efficiency, we can reorganize the matrix M into a structure known as Stripes (Figure 2), where M is partitioned into k horizontal segments. Each stripe M_i contains only the edges whose destination nodes belong to the i-th block of r^{new}. Derive the improved total I/O cost per iteration in terms of $|M|$, $|r^{old}|$, $|r^{new}|$, k, and ϵ. Here, $\epsilon > 0$ is overhead due to duplication of source and degree, indicating that the stripe representation requires additional space proportional to $|M|$.

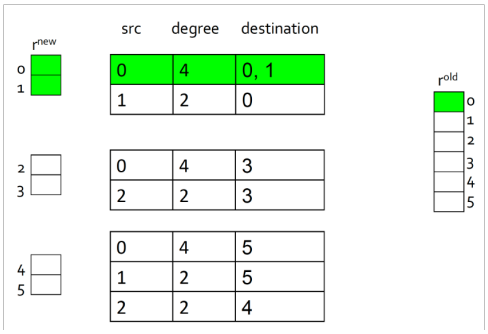


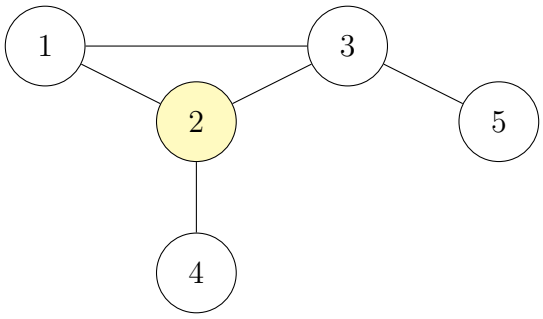
Figure 2: Stripe-Based Update

$$(1+\epsilon)|M| + k(|r^{old}|) + |r^{new}|$$

4 [20 points] Graph Representation Learning

Consider the undirected graph $G = (V, E)$ with

$$V = \{1, 2, 3, 4, 5\}, \qquad E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 5)\}.$$



(a) Suppose we run a biased random walk on the graph G. In this biased random walk, we use the node2vec bias parameters

$$p = \tfrac{1}{2}, \qquad q = 2,$$

and define the unnormalized transition weight $\alpha_{p,q}(t, x)$ as

$$\alpha_{p,q}(t, x) = \begin{cases} 1/p, & \text{if } d(t, x) = 0, \\ 1, & \text{if } d(t, x) = 1, \\ 1/q, & \text{if } d(t, x) = 2, \end{cases}$$

where $d(t, x)$ is the shortest-path distance between the previous node t and the candidate next node x. At each step, from current node v (with previous node t), the transition probability to a neighbor x of v is proportional to $\alpha_{p,q}(t, x)$. For the *very first* step from the root, we assume a uniform choice among the neighbors of the root node.

Fix the root node to be $r = 2$ and consider a biased random walk of length 2, which consists of three nodes, starting at node 2.

(i) List all possible length-2 walks

$$N = (N[1], N[2], N[3])$$

starting from $N[1] = 2$ on this graph.

$$(2, 1, 2),\ (2, 1, 3),\ (2, 3, 1),\ (2, 3, 2),\ (2, 3, 5),\ (2, 4, 2).$$

(ii) For each walk obtained from (i), compute its probability under the biased random walk defined above. Also, among these walks, find the walk $N^\star$ with the highest probability (Hint: Transition weight should be normalized).

We use the node2vec bias

$$p = \tfrac{1}{2}, \qquad q = 2,$$

and

$$\alpha_{p,q}(t,x) = \begin{cases} 1/p = 2, & d(t,x) = 0, \\ 1, & d(t,x) = 1, \\ 1/q = 1/2, & d(t,x) = 2. \end{cases}$$

For the first step from node 2 we already fixed $\mathbb{P}(2 \to 1) = \mathbb{P}(2 \to 3) = \mathbb{P}(2 \to 4) = \frac{1}{3}$. For the second step, from current node v with previous node $t = 2$, we assign to each neighbor x of v a weight $\alpha_{p,q}(2,x)$ and normalize.

<u>Case 1: $2 \to 1 \to \cdot$.</u>

$$\alpha(2,2) = 2, \qquad \alpha(2,3) = 1.$$

The normalized probabilities are

$$\mathbb{P}(1 \to 2 \mid \text{prev } 2) = \frac{2}{2+1} = \frac{2}{3}, \qquad \mathbb{P}(1 \to 3 \mid \text{prev } 2) = \frac{1}{3}.$$

Therefore

$$\mathbb{P}\big((2,1,2)\big) = \frac{1}{3} \cdot \frac{2}{3} = \frac{2}{9}, \qquad \mathbb{P}\big((2,1,3)\big) = \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{9}.$$

<u>Case 2: $2 \to 3 \to \cdot$.</u>

$$\alpha(2,1) = 1, \qquad \alpha(2,2) = 2, \qquad \alpha(2,5) = 1/2.$$

hence

$$\mathbb{P}(3 \to 1 \mid \text{prev } 2) = \frac{2}{7}, \quad \mathbb{P}(3 \to 2 \mid \text{prev } 2) = \frac{4}{7}, \quad \mathbb{P}(3 \to 5 \mid \text{prev } 2) = \frac{1}{7}.$$

Therefore

$$\mathbb{P}\big((2,3,1)\big) = \frac{1}{3} \cdot \frac{2}{7} = \frac{2}{21}, \qquad \mathbb{P}\big((2,3,2)\big) = \frac{1}{3} \cdot \frac{4}{7} = \frac{4}{21}, \qquad \mathbb{P}\big((2,3,5)\big) = \frac{1}{3} \cdot \frac{1}{7} = \frac{1}{21}.$$

<u>Case 3: $2 \to 4 \to \cdot$.</u>

$$\mathbb{P}\big((2,4,2)\big) = \frac{1}{3}.$$

The most likely walk is

$$N^\star = (2,4,2),$$

with probability $\mathbb{P}(N^\star) = \frac{1}{3}$.

(b) We now use the neighborhood sequence $M = (2, 3, 5)$ as the input to a simplified node2vec update.

Each node $i \in \{1, 2, 3, 4, 5\}$ has a one-dimensional embedding $z_i \in \mathbb{R}$, and we collect them into

$$Z = (z_1, z_2, z_3, z_4, z_5)^\top.$$

We use the following loss between two nodes u and v:

$$L(u, v) = \frac{1}{2}(z_u - z_v)^2,$$

and a learning rate $\eta = \frac{1}{2}$.

The simplified node2vec update with a single root $r = 2$, a fixed sequence M, and window size $w = 1$ is:

1. Initialize the embeddings Z.
2. Fix the neighborhood sequence $M = (M[1], M[2], M[3])$.
3. Fix the window size $w = 1$.
4. For each position $i \in \{1, 2, 3\}$:
   4.1. Let $u = M[i]$ be the *center node*.
   4.2. For each position j with $1 \le j \le 3$, $j \ne i$, and $|j - i| \le 1$:
      4.2.1. Let $v = M[j]$ be a *context node* and update

$$Z \leftarrow Z - \eta \nabla_Z L(u, v).$$

(i) For a single pair (u, v), show that one gradient-descent step with learning rate $\eta = \frac{1}{2}$ updates both embeddings to their arithmetic mean:

$$z'_u = z'_v = \frac{z_u + z_v}{2}.$$

The loss

$$L(u, v) = \frac{1}{2}(z_u - z_v)^2$$

has partial derivatives

$$\frac{\partial L}{\partial z_u} = \frac{1}{2} \cdot 2(z_u - z_v) = z_u - z_v, \qquad \frac{\partial L}{\partial z_v} = \frac{1}{2} \cdot 2(z_v - z_u) = z_v - z_u.$$

A gradient-descent step with learning rate $\eta = \frac{1}{2}$ gives

$$z'_u = z_u - \eta \frac{\partial L}{\partial z_u} = z_u - \frac{1}{2}(z_u - z_v) = \frac{1}{2}z_u + \frac{1}{2}z_v = \frac{z_u + z_v}{2},$$

$$z'_v = z_v - \eta \frac{\partial L}{\partial z_v} = z_v - \frac{1}{2}(z_v - z_u) = \frac{1}{2}z_v + \frac{1}{2}z_u = \frac{z_u + z_v}{2}.$$

Thus, after one update, both embeddings are equal to their arithmetic mean:

$$z'_u = z'_v = \frac{z_u + z_v}{2}.$$

(ii) We fix the initial embeddings to be

$$z_1^{(0)} = 1, \quad z_2^{(0)} = 2, \quad z_3^{(0)} = 3, \quad z_4^{(0)} = 4, \quad z_5^{(0)} = 5.$$

Using the neighborhood sequence M, compute the final embeddings

$$z_1^{(\text{final})},\ z_2^{(\text{final})},\ z_3^{(\text{final})},\ z_4^{(\text{final})},\ z_5^{(\text{final})}$$

after the simplified node2vec update rule described above. Note that every *for* loop in the algorithm is executed from left to right, and for each (center, context) pair you should perform *only one* gradient-descent update as in part (i).

We have the neighborhood sequence

$$M = (M[1], M[2], M[3]) = (2, 3, 5)$$

and window size $w = 1$.

Center–context pairs. For each position $i \in \{1, 2, 3\}$, we set $u = M[i]$ as the center node. For all indices j with $1 \le j \le 3$, $j \ne i$, and $|j - i| \le 1$, we set $v = M[j]$ as a context node.

- $i = 1$: center $u = M[1] = 2$. The only valid index is $j = 2$, so we get the pair $(u, v) = (2, 3)$.
- $i = 2$: center $u = M[2] = 3$. Now $j = 1$ and $j = 3$ satisfy $|j - 2| \le 1$ and $j \ne 2$, so we obtain the pairs $(3, 2)$ and $(3, 5)$.
- $i = 3$: center $u = M[3] = 5$. The only valid index is $j = 2$, giving the pair $(5, 3)$.

Thus, in order, the algorithm performs gradient updates for the pairs

$$(2, 3),\ (3, 2),\ (3, 5),\ (5, 3).$$

Sequential updates of embeddings. We start from

$$(z_1^{(0)}, z_2^{(0)}, z_3^{(0)}, z_4^{(0)}, z_5^{(0)}) = (1, 2, 3, 4, 5).$$

For each pair (u, v), by part (i) we replace z_u and z_v by the arithmetic mean of their *current* values.

<u>Step 1: pair $(u, v) = (2, 3)$.</u>

Currently $z_2^{(0)} = 2$ and $z_3^{(0)} = 3$, so their mean is

$$m_1 = \frac{2 + 3}{2} = 2.5.$$

After the update,

$$z_2^{(1)} = z_3^{(1)} = 2.5,$$

and the other embeddings remain the same:

$$(z_1^{(1)}, z_2^{(1)}, z_3^{(1)}, z_4^{(1)}, z_5^{(1)}) = (1,\ 2.5,\ 2.5,\ 4,\ 5).$$

Step 2: pair $(u, v) = (3, 2)$.

Now $z_2^{(1)} = 2.5$ and $z_3^{(1)} = 2.5$, so the mean is still 2.5 and nothing changes:

$$(z_1^{(2)}, z_2^{(2)}, z_3^{(2)}, z_4^{(2)}, z_5^{(2)}) = (1,\ 2.5,\ 2.5,\ 4,\ 5).$$

Step 3: pair $(u, v) = (3, 5)$.

Now $z_3^{(2)} = 2.5$ and $z_5^{(2)} = 5$, so the mean is

$$m_2 = \frac{2.5 + 5}{2} = 3.75.$$

After the update,

$$z_3^{(3)} = z_5^{(3)} = 3.75,$$

and other nodes are unchanged:

$$(z_1^{(3)}, z_2^{(3)}, z_3^{(3)}, z_4^{(3)}, z_5^{(3)}) = (1,\ 2.5,\ 3.75,\ 4,\ 3.75).$$

Step 4: pair $(u, v) = (5, 3)$.

Now $z_3^{(3)} = z_5^{(3)} = 3.75$, so one more update does not change the values:

$$(z_1^{(4)}, z_2^{(4)}, z_3^{(4)}, z_4^{(4)}, z_5^{(4)}) = (1,\ 2.5,\ 3.75,\ 4,\ 3.75).$$

Therefore, after one pass of the simplified node2vec update, the final embeddings are

$$\boxed{z_1^{(\text{final})} = 1, \quad z_2^{(\text{final})} = 2.5, \quad z_3^{(\text{final})} = 3.75, \quad z_4^{(\text{final})} = 4, \quad z_5^{(\text{final})} = 3.75.}$$

5 [20 points] Graph Neural Networks

(a) [12 points] Consider an undirected graph constructed from an KAIST community:

- nodes V: $p \in V$ if p is a KAIST student;
- edges E: $(p, q) \in E$ if p and q are enrolled in at least one common course;

Each node p has a label $y(p)$ indicating the student's major (e.g., "EE"). You may assume that students with the same major are more likely to take similar courses.

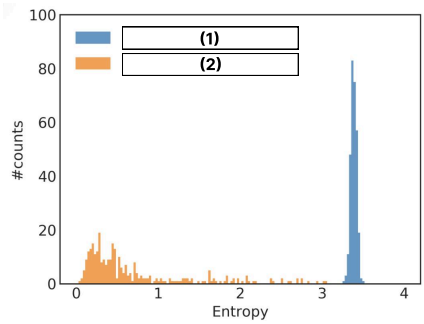


Figure (i).

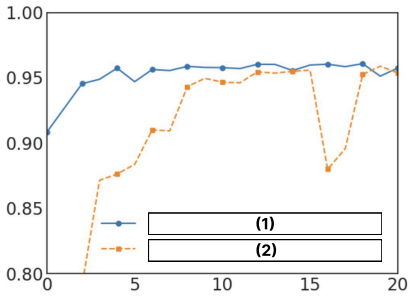


Figure (ii).

(i) [6 points] Using only the edges E, we partition the nodes into 300 disjoint groups ("partitions") with two different partitioning algorithms:

- random partition
- clustering-based partition

The Figure (i) shows the histograms of label-distribution entropy values for the 300 partitions obtained from each algorithm.

Match the labels (1) and (2) to the corresponding algorithms above. Then, provide a proper explanation of what the figure indicates in one sentence.

Note: For each partition S, we compute the label-distribution entropy, $H(S) = -\sum_c p_c \log_2 p_c$, where p_c is the fraction of nodes in S whose label is c.

Answer. (1): random partition, (2): clustering-based partition

Clustering-based partitions are not diverse enough to represent the entire graph.

Criterion.

- 2 points for (1) and (2). 2 points for interpretation.

(ii) [6 points] The Figure (ii) shows the F1 score versus training epoch for two batching strategies used during the training of Cluster-GCN:

- multiple clusters per batch
- a single cluster per batch

Match the labels (1) and (2) to the corresponding strategies above. Then, provide a proper explanation of what the figure indicates in one sentence.

Note: F1 score measures node-classification performance; higher values indicate better prediction.

Answer. (1): multiple clusters per batch, (2): a single cluster per batch

Using a single cluster per batch yields slow convergence of training.

Criterion.

- 2 points for (1) and (2). 2 points for interpretation.

(b) [8 points] Recall how the final embeddings of nodes are generated from LightGCN:

$$\mathbf{Z} = \alpha_0 \mathbf{H}^{(0)} + \alpha_1 \mathbf{H}^{(1)} + \cdots + \alpha_L \mathbf{H}^{(L)},$$

where $\mathbf{H}^{(l+1)} = \tilde{\mathbf{A}}\mathbf{H}^{(l)}$, $\tilde{\mathbf{A}} = \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ and $\mathbf{D}$ is the degree matrix of $\mathbf{A}$.
In this problem, we aim to show that this multi-scale diffusion can be reproduced without explicitly storing every $\mathbf{H}^{(l)}$, by defining new node embeddings $\mathbf{E}^{(l)}$ and an appropriate update function. Consider the following GNN layer:

$$\mathbf{E}^{(l+1)} = \text{UPDATE}^{(l)}(\tilde{\mathbf{A}}\mathbf{E}^{(l)}, \mathbf{E}^{(l)}),$$

with the initialization $\mathbf{E}^{(0)} = \mathbf{H}^{(0)}$.
Given $\{\alpha_i\}$ and L, design $\{\text{UPDATE}^{(l)}(\cdot,\cdot)\}$ for $l = 0, \cdots, L-1$ so that, after L iterations of update, the recursion produces the weighted aggregation:

$$\mathbf{E}^{(L)} = \mathbf{Z} = \textstyle\sum_{i=0}^{L} \alpha_i \mathbf{H}^{(i)}$$

That is, the recursion on $\mathbf{E}^{(l)}$ exactly simulates the multi-scale diffusion over all $\mathbf{H}^{(l)}$.

(i) [4 points] $\alpha_i = \frac{1}{L+1}$.

Answer.

$$\text{UPDATE}^{(l)}(\tilde{\mathbf{A}}\mathbf{E}^{(l)}, \mathbf{E}^{(l)}) = \begin{cases} \left[\tilde{\mathbf{A}}\mathbf{E}^{(l)},\ \frac{1}{L+1}\mathbf{E}^{(l)}\right. & l = 0 \\ \left[\tilde{\mathbf{A}}\mathbf{E}^{(l)}_{(1)},\ \mathbf{E}^{(l)}_{(2)} + \frac{1}{L+1}\mathbf{E}^{(l)}_{(1)}\right] & l = 1, \cdots, L-2 \\ \mathbf{E}^{(l)}_{(2)} + \frac{1}{L+1}\mathbf{E}^{(l)}_{(1)} + \frac{1}{L+1}\tilde{\mathbf{A}}\mathbf{E}^{(l)}_{(1)} & l = L-1 \end{cases}$$

Note that this cannot be achieved with a single d-dimensional form:

$$\mathbf{E}^{(l+1)} = a_l \tilde{\mathbf{A}}\mathbf{E}^{(l)} + b_l \mathbf{E}^{(l)}.$$

Criterion.

- Deduct points if there is a minor mistake.

(ii) [4 points] $\alpha_i = \binom{L}{i}(1-\beta)^{L-i}\beta^i$.

Answer. $\text{UPDATE}^{(l)}(\tilde{\mathbf{A}}\mathbf{E}^{(l)}, \mathbf{E}^{(l)}) = \beta\tilde{\mathbf{A}}\mathbf{E}^{(l)} + (1-\beta)\mathbf{E}^{(l)}$.

Criterion.

- Deduct points if there is a minor mistake.

6 [20 points] Mining Data Streams

In many data mining applications, we process a continuous stream of high-dimensional data rows $a_1, a_2, \cdots \in \mathbb{R}^d$. We often wish to perform principal component analysis (PCA) to find the most important *directions* in the data. However, in a streaming setting, we cannot store the entire history of data $A = [a_1, a_2, \cdots]$. The **frequent directions (FD)** algorithm solves this problem by maintaining a small sketch matrix $B \in \mathbb{R}^{\ell \times d}$. The goal is for $B^T B$ to approximate the true covariance $C = A^T A$ closely.

The FD algorithm balances accuracy with efficiency using a buffer system:

1. **Accumulate:** The algorithm maintains a sketch matrix B of size $\ell \times d$. As new rows arrive, they are simply appended to B.
2. **Buffer Full:** When B reaches double its desired size (e.g., 2ℓ rows), it runs out of memory and must be compressed back to size ℓ.
3. **Select Pivot (δ):** To reduce the rank from 2ℓ back to ℓ, we calculate the singular value decomposition (SVD) of B. We identify the $(\ell+1)$-th squared singular value (the median energy) and set this as our shrinkage value δ.
4. **Shrink:** We subtract δ from *every* squared singular value: $\tilde{s}_i^2 = \max(0, s_i^2 - \delta)$.

By setting δ to the $(\ell+1)$-th value, we ensure that at least ℓ singular values become zero (since $s_{\ell+1}^2 - \delta = 0$). This allows us to discard those directions and physically reduce the matrix size back to ℓ rows.

In this problem, we will first analyze the error introduced by this shrinkage mechanism, and then extend it to a non-stationary environment where old data decays over time.

(a) [4 points] The Covariance Error

Let $B \in \mathbb{R}^{m \times d}$ be a matrix with singular value decomposition $B = USV^T$, where $S = \text{diag}(s_1, \ldots, s_m)$.

We create a "shrunk" matrix $\tilde{B}$ by reducing the squared singular values by a value $\delta \geq 0$. Specifically, the new singular values are $\tilde{s}_i = \sqrt{\max(0, s_i^2 - \delta)}$.

Prove that the spectral norm of the error between the original covariance and the shrunk covariance is bounded by δ.

That is, show that:

$$\|B^T B - \tilde{B}^T \tilde{B}\|_2 \leq \delta$$

Answer:

Let $D = B^T B - \tilde{B}^T \tilde{B}$. Since the eigenvectors V are preserved during shrinkage, we can express the difference as:

$$D = VS^2V^T - V\tilde{S}^2V^T = V(S^2 - \tilde{S}^2)V^T$$

The spectral norm of D is given by the maximum absolute value of the diagonal entries of the matrix $(S^2 - \tilde{S}^2)$. Let's analyze the i-th diagonal entry, $s_i^2 - \tilde{s}_i^2$:

- **Case 1 ($s_i^2 \geq \delta$):** The new value is $\tilde{s}_i^2 = s_i^2 - \delta$. The difference is $s_i^2 - (s_i^2 - \delta) = \delta$.
- **Case 2 ($s_i^2 < \delta$):** The new value is $\tilde{s}_i^2 = 0$. The difference is $s_i^2 - 0 = s_i^2$. Since $s_i^2 < \delta$, this difference is strictly less than δ.

In all cases, the difference is at most δ. Therefore:

$$\|B^T B - \tilde{B}^T \tilde{B}\|_2 = \max_i |s_i^2 - \tilde{s}_i^2| \leq \delta$$

(b) [8 points] Recurrence with Ghost Decay

In non-stationary environments, we want to emphasize recent data by decaying the importance of older history.

To achieve this without storing timestamps, we use "ghost" decay. Recall that our sketch B approximates the covariance as $B^T B \approx C$. If we want to decay the old covariance by a factor γ, we can simply scale the sketch matrix B by $\sqrt{\gamma}$:

$$(\sqrt{\gamma}B)^T(\sqrt{\gamma}B) = \gamma(B^T B) \approx \gamma C$$

Based on this, the algorithm proceeds as follows at each time step t:

(1) **Decay:** Scale the previous sketch: $B \leftarrow \sqrt{\gamma}B_{t-1}$.
(2) **Insert:** Append the new row a_t to B.
(3) **Maintain:** Check the size of B.
   - If B is full (e.g., reaches size 2ℓ), perform the shrinkage operation to reduce its size back to ℓ.
   - If B is not full, do nothing ($\delta_t = 0$).

Let $C_{(t)} = \gamma C_{(t-1)} + a_t a_t^T$ be the true time-decayed covariance of the stream data at time t. Let $E_t = \|C_{(t)} - B_t^T B_t\|_2$ be the approximation error at time t.

Prove that the error satisfies the following recurrence relation:

$$E_t \leq \gamma E_{t-1} + \delta_t$$

Answer:

By definition, the error at time t is $E_t = \|C_{(t)} - B_t^T B_t\|_2$. Substituting the recursive definition of $C_{(t)}$, we get:

$$E_t = \|(\gamma C_{(t-1)} + a_t a_t^T) - B_t^T B_t\|_2$$

Let M_t be the intermediate matrix formed by stacking the decayed history and the new row: $M_t = \begin{bmatrix} \sqrt{\gamma}B_{t-1} \\ a_t \end{bmatrix}$. Note that $M_t^T M_t = \gamma B_{t-1}^T B_{t-1} + a_t a_t^T$. The algorithm obtains B_t by performing shrinkage on M_t with removed energy δ_t. From part (a), we know that the error introduced by this shrinkage is bounded by δ_t:

$$\|M_t^T M_t - B_t^T B_t\|_2 \leq \delta_t$$

Now, we rewrite E_t by adding and subtracting $M_t^T M_t$ inside the norm:

$$E_t = \|(\gamma C_{(t-1)} + a_t a_t^T) - M_t^T M_t + M_t^T M_t - B_t^T B_t\|_2$$

Substituting $M_t^T M_t = \gamma B_{t-1}^T B_{t-1} + a_t a_t^T$, the first part simplifies:

$$(\gamma C_{(t-1)} + a_t a_t^T) - (\gamma B_{t-1}^T B_{t-1} + a_t a_t^T) = \gamma(C_{(t-1)} - B_{t-1}^T B_{t-1})$$

Thus:

$$E_t = \|\gamma(C_{(t-1)} - B_{t-1}^T B_{t-1}) + (M_t^T M_t - B_t^T B_t)\|_2$$

Using the Triangle Inequality ($\|X + Y\| \leq \|X\| + \|Y\|$):

$$E_t \leq \gamma\|C_{(t-1)} - B_{t-1}^T B_{t-1}\|_2 + \|M_t^T M_t - B_t^T B_t\|_2$$

$$E_t \leq \gamma E_{t-1} + \delta_t$$

(c) [8 points] Asymptotic Bound

Assume the stream continues infinitely ($t \to \infty$) and the rows are bounded such that $\|a_t\|^2 \leq 1$.

Let's consider a *theoretical row-by-row variation* of the ghost decay FD algorithm that performs a shrinkage operation at *every* time step regardless of the current buffer size.

In this variation, the rank is constantly reduced from $\ell+1$ to ℓ, meaning the removed energy δ_t corresponds to the smallest squared singular value.

1. Explain why the removed energy at any step is bounded by $\delta_t \leq \frac{1}{\ell(1-\gamma)}$.

2. Using the recurrence relation from part (b), derive the asymptotic upper bound on the error E_∞.
3. How does the sketch size ℓ relate to the decay rate γ if we want to keep the error bounded by a constant ϵ?

Answer:

1. The removed energy δ_t is the smallest squared singular value of the augmented sketch (rank $\ell + 1$). The smallest eigenvalue cannot exceed the average eigenvalue. The sum of eigenvalues is the Trace. Thus:

$$\delta_t \leq \frac{\text{Trace}(C_{(t)})}{\ell + 1} < \frac{\text{Trace}(C_{(t)})}{\ell}$$

Since $\|a_i\|^2 \leq 1$, the Trace of the decayed covariance converges as $t \to \infty$:

$$\text{Trace}(C_{(\infty)}) \leq \sum_{i=0}^{\infty} \gamma^i = \frac{1}{1-\gamma}$$

Therefore, $\delta_t \leq \frac{1}{\ell(1-\gamma)}$.

2. Using the recurrence $E_t \leq \gamma E_{t-1} + \delta_t$, we expand it recursively:

$$E_t \leq \delta_t + \gamma\delta_{t-1} + \gamma^2\delta_{t-2} + \dots$$

Substituting the upper bound $\delta_{max} = \frac{1}{\ell(1-\gamma)}$ for all terms:

$$E_\infty \leq \delta_{max} \sum_{k=0}^{\infty} \gamma^k = \delta_{max} \cdot \frac{1}{1-\gamma}$$

$$E_\infty \leq \left(\frac{1}{\ell(1-\gamma)}\right)\left(\frac{1}{1-\gamma}\right) = \frac{1}{\ell(1-\gamma)^2}$$

3. To ensure the error is bounded by ϵ, we require:

$$\frac{1}{\ell(1-\gamma)^2} \leq \epsilon \implies \ell \geq \frac{1}{\epsilon(1-\gamma)^2}$$